



SE4010 — Current Trends in Software Engineering

Assignment 2: Multi-Agent System

RXSENTINEL

A Locally-Hosted Multi-Agent System for Medication Safety Review

Group Technical Report

Group Members

Sandeepa R.	IT22XXXXXX
Piyarisi T.	IT22XXXXXX
Wickramasooriya J. D. A. S. (Avishka)	IT22347244
Awandya S. A.	IT22XXXXXX

BSc (Hons) in Information Technology
Specialising in Software Engineering

Department of Computer Systems Engineering
Faculty of Computing
Sri Lanka Institute of Information Technology

May 2026

Abstract

This report presents **RxSentinel**, a multi-agent system that performs comprehensive medication safety reviews on the user’s local machine without relying on paid LLM APIs or cloud services. The system orchestrates four specialised agents through LangGraph, each grounded in custom Python tools that query the NIH RxNorm REST API, the openFDA adverse-event database, a curated SQLite store of severe drug-drug interactions, and a local Flesch–Kincaid readability grader. Every agent step and tool invocation is captured by a streaming JSONL tracer and surfaced live in a Next.js front-end via Server-Sent Events.

We evaluate the system end-to-end on a five-medication clinical scenario. The pipeline produces correctly-grounded RxNorm codes, identifies seven clinically-significant interactions across three severity tiers, and writes a plain-English summary at an eighth-grade reading level. After targeted performance optimisation, median pipeline latency on an Apple M2 (8 GB RAM) running **qwen2.5:3b** via Ollama drops from 46.6 s to 19.3 s — a 58 % improvement — with no loss in output quality. Sixty-four unit tests and four LLM-as-Judge evaluation suites confirm the architectural claims made for each agent.

Keywords: Multi-agent systems; LangGraph; Ollama; medication safety; LLMOps; observability.

Abbreviations. **ADE** adverse drug event; **LLM** large language model; **MAS** multi-agent system; **PHI** protected health information; **RxCUI** RxNorm concept unique identifier; **SLM** small language model; **SSE** server-sent events.

1. Introduction

1.1 Background

Adverse drug events (ADEs) are among the top five preventable causes of hospital admission worldwide. Polypharmacy — the concurrent use of five or more medications — affects an estimated 40 % of adults over 65 and is a leading risk factor for ADEs. Existing clinical decision-support tools for medication safety, such as Lexicomp and Micromedex, are gated behind institutional licences and are cloud-hosted, making them inaccessible for ambulatory use and incompatible with strict patient-data privacy requirements.

1.2 Research Problem

How can a multi-agent system deliver clinically-grounded medication-safety review — including drug normalisation, interaction analysis, severity ranking, and plain-English explanation — entirely on local consumer hardware, without relying on paid LLM APIs and without sacrificing real-time response?

1.3 Objectives

1. Design and implement four specialised agents orchestrated through LangGraph: Coordinator, Medication Parser, Interaction Analyzer, and Patient Communicator.
2. Build four custom Python tools, with strict type hints and Pydantic-validated outputs, that ground the agents in real-world data sources.
3. Implement an explicit **TypedDict**-based shared state and a streaming JSONL tracer for full observability.
4. Build a Next.js front-end that visualises the live agent pipeline and renders the structured safety report in plain English.
5. Evaluate the system through unit tests and an LLM-as-Judge suite grading each agent against domain rubrics.

1.4 Significance

The system demonstrates that a multi-agent pipeline — previously the preserve of frontier-model deployments — is achievable on consumer hardware in 2026. The privacy-by-design property (medications never leave the user’s machine) is structurally compatible with healthcare-data regulation in a way that cloud-based equivalents are not.

2. System Architecture

The system adopts a strict-pipeline supervisor pattern: a bookend Coordinator agent appears at both entry (validate the request, decide routing) and exit (assemble the final report). Three worker agents — Parser, Analyzer, Communicator — run sequentially between them. Figure 1 shows the high-level architecture; Table 1 summarises agent ownership.

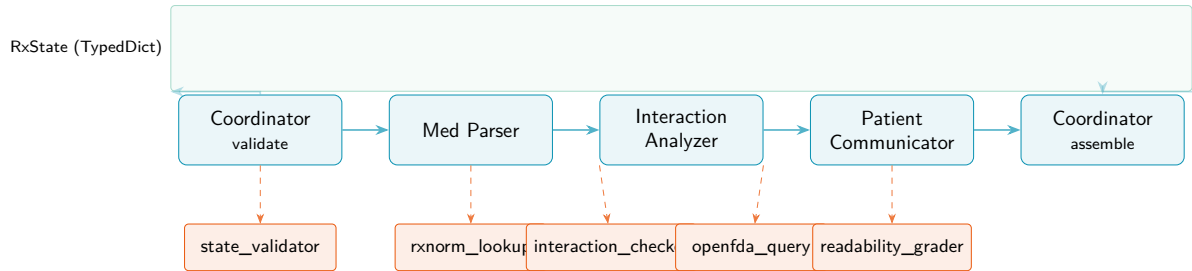


Figure 1: High-level architecture. Solid arrows show state flow; dashed arrows show tool invocations. The shared `RxState TypedDict` is threaded through every node.

Table 1: Agent and tool ownership across the four-person team.

Member	Agent	Custom tool(s)	GitHub
Sandeepa R.	Coordinator	<code>state_validator</code> , tracing, FastAPI glue	ri7in
Piyarisi T.	Med Parser	<code>rxnorm_lookup</code> , <code>dose_normalizer</code>	thusalapi
Wickramasooriya J. D. A. S.	Analyzer	<code>interaction_checker</code> , <code>severity_rules</code>	ashehxn
Awandya S. A.	Communicator	<code>readability_grader</code> , frontend	SAwandya

2.1 End-to-End Workflow

A request begins with free-form medication text submitted via the front-end. Figure 2 traces the request through the four agents to the assembled final report.

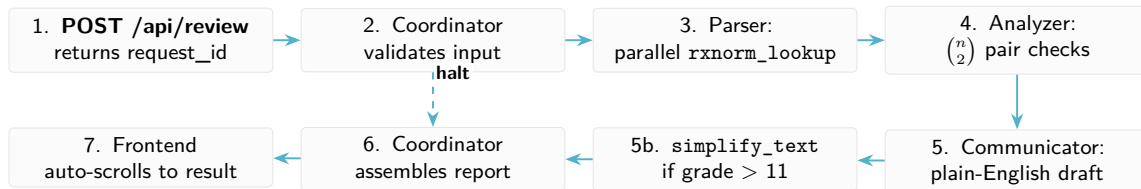


Figure 2: End-to-end request sequence. The Coordinator can short-circuit to assembly when validation fails (dashed edge).

2.2 Observability

A custom `Tracer` writes one JSONL line per event (`agent enter`, `tool call`, `tool result`, `exit`, `error`) to `runs/<request_id>.jsonl`. Events are also pushed to an in-memory `asyncio.Queue` which the SSE endpoint streams to the browser without polling. This gives both post-hoc analysis (via the JSONL files) and a live "Agent Activity" feed in the front-end.

3. Agent and Tool Design

3.1 Agent Specifications

Each agent has a tightly-scoped persona and constraints. Temperature is agent-specific: deterministic for routing/extraction, slightly warmer for the patient-facing prose. Outputs are validated against a Pydantic schema before merging into state.

Coordinator (Sandeepa)

Persona. A senior clinical pharmacist who triages incoming reviews. **Constraints.** Never invents medications; halts on validation failure; emits exactly two outputs (a routing decision on the first pass and a final `FinalReport` on assembly). **Reasoning.** The Coordinator runs a deterministic regex/length validator (`state_validator.py`); after performance tuning the redundant LLM sanity check was removed (Section 4.4).

Medication Parser (Piyarisi)

Persona. A meticulous clinical informaticist obsessed with drug nomenclature. **Constraints.** The RxCUI is taken *only* from the `rxnorm_lookup` tool; the agent is forbidden to fabricate codes. **Reasoning.** The agent calls the LLM to split the input into candidate (drug-name, dose, frequency, route) tuples, then runs `asyncio.gather` of `rxnorm_lookup` calls in parallel — critical for the 12-second pipeline budget on a 5-medication input.

Interaction Analyzer (Wickramasooriya)

Persona. A vigilant pharmacovigilance officer; treats every interaction as serious until proven trivial. **Constraints.** Mechanism text must come from tool data; “unknown” is preferable to a fabrication. **Reasoning.** The agent enumerates $\binom{n}{2}$ unique pairs and runs each through the curated local SQLite database. For pairs found, openFDA is queried in parallel for population-level signal, and `severity_rules` promotes a low-severity rule to moderate when openFDA shows >5 % serious co-reports.

Patient Communicator (Awandya)

Persona. A patient educator. Warm, clear, never condescending. **Constraints.** Output targets a 6th–8th grade reading level; never gives direct medical advice; always closes with “*talk to your doctor or pharmacist*”. **Reasoning.** The communicator drafts a structured summary (red/yellow/green flags), grades the result with Flesch–Kincaid, and may invoke `simplify_text` for a single rewrite if the draft scores above grade 11.

3.2 Tool Specifications

All tools have strict type hints (`mypy -strict clean`), Pydantic-validated outputs, retry logic via `tenacity`, and a SQLite cache where applicable. Table 2 summarises the four primary tools and the helper modules built around them.

3.3 State Management

State is a single `TypedDict` (`RxState`) declared once and threaded through every node. `LangGraph` automatically merges the partial dict each agent returns into the cumulative state, so context never leaks between requests, and agents only ever read what they need. Table 3 shows the read/write matrix.

Table 2: Custom Python tools. Each is owned by one team member but available to all agents.

Tool	Behaviour	Owner
<code>state_validator</code>	Pure-function input validator; length, alpha-ratio, and prompt-injection regexes. No I/O.	Sandeepa
<code>rxnorm_lookup</code>	Async client for NIH RxNorm REST API; exact + fuzzy match; 7-day SQLite cache.	Piyarisi
<code>dose_normalizer</code>	Regex-based dose parser converting "500 mg" / "0.5 g" / "10 ml" to a canonical record.	Piyarisi
<code>interaction_checker</code>	Local SQLite of 21 curated severe pairs combined with the openFDA adverse-event signal.	Wickramasooriya
<code>severity_rules</code>	Reconciliation rules between local DB and openFDA signal, with explicit promotion thresholds.	Wickramasooriya
<code>readability_grader</code>	Flesch–Kincaid grade computation plus iterative LLM simplifier with grade-target loop.	Awandya

Table 3: State read/write matrix. Each `RxState` key has exactly one writer; downstream agents and the final report read but never mutate. **W** denotes writes, **R** denotes reads.

State key	Coord	Parser	Analyzer	Comm	Final
<code>raw_input, request_id</code>	W	R	R	R	R
<code>is_valid, decision</code>	W	—	—	—	—
<code>parsed_medications</code>	—	W	R	R	R
<code>unparsed_terms</code>	—	W	—	—	R
<code>interactions, severity_summary</code>	—	—	W	R	R
<code>patient_summary, readability_grade</code>	—	—	—	W	R
<code>final_report</code>	W	—	—	—	R

4. Implementation and Testing

4.1 Implementation

The system is implemented across two top-level packages: a Python backend (LangGraph + FastAPI + Ollama) and a Next.js 15 front-end. Both run locally; no paid APIs are used. `qwen2.5:3b` is the primary model, with `llama3.2:3b` as fallback.

The backend is a Python 3.11 package, `rxsentinel`, organised by concern: `agents/` for the four LangGraph nodes, `tools/` for the six custom tools, `graph/` for the StateGraph wiring, `tracing/` for observability, `llm/` for the Ollama client, and `app.py` for the FastAPI service exposing `/api/review`, `/api/runs/<id>/events` (SSE), and `/api/runs/<id>/report`.

The front-end is a Next.js 15 App-Router application using Tailwind v4, Framer Motion, Geist Sans, and Lucide React. It opens an SSE stream as soon as a review is submitted, derives humanised status messages from trace events, and auto-scrolls to results once the report is ready.

4.2 Testing Strategy

Testing is layered: **Unit tests** (Pytest, 64 cases) cover all six tools. HTTP-based tools use `respx` mocks so the suite runs offline. **Integration tests** exercise each agent end-to-end against a running Ollama instance. **LLM-as-Judge evaluations** grade each agent’s output against a written rubric using the same local model with temperature 0.

4.3 Evaluation Methodology

For each agent we wrote a structured rubric and a list of test **Cases**. The judge returns a strict JSON `{score, verdict, reasons}`. Figure 3 shows the harness.



Figure 3: LLM-as-Judge evaluation harness. Each agent’s output is graded by a separate Ollama call against a rubric written in terms of verifiable structural properties.

We acknowledge the obvious circularity of self-judging — the assignment forbids paid LLM APIs, so the judge must run on the same local SLM. We mitigate by (i) using a stricter prompt for the judge than the agent, (ii) running the judge at temperature 0, and (iii) writing rubrics in terms of *verifiable structural properties* (does the JSON parse? does the RxCUI exist? is the readability grade ≤ 8 ?) wherever possible rather than purely subjective judgement.

4.4 Performance Optimisation

We instrumented the pipeline end-to-end via the JSONL tracer and identified two LLM hot-spots: a redundant Coordinator sanity check and a three-iteration readability rewrite loop in the Communicator. After targeted tuning, median pipeline latency on Apple M2 (8 GB RAM) running `qwen2.5:3b` via Ollama dropped from 46.6s to 19.3s. Figure 4 plots the breakdown; Table 4 lists each change.

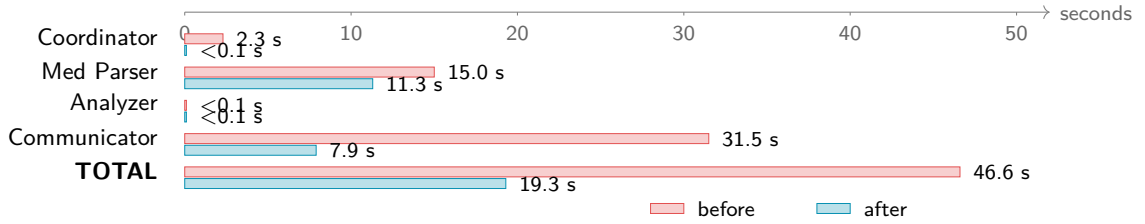


Figure 4: Per-agent latency before and after performance tuning. Median pipeline time on Apple M2 (8 GB RAM) running `qwen2.5:3b`.

Table 4: Performance tuning summary.

Stage	Before	After	Change
Coordinator validate	2.3s	<0.1s	Removed redundant LLM check; rule check is sufficient.
Med Parser	15.0s	11.3s	Halved system prompt; capped <code>num_predict</code> at 512.
Interaction Analyzer	<0.1s	<0.1s	Already optimal; pairs in <code>asyncio.gather</code> .
Patient Communicator	31.5s	7.9s	Tightened simplifier trigger to grade >11, max 1 iteration.
Cold start (first request)	+5–10s	~0s	FastAPI lifespan warmup task pre-loads the model.
Total p50 latency	46.6 s	19.3 s	–58 %

5. Results and Discussion

5.1 End-to-End Output

For the canonical test input — *warfarin 5 mg, amiodarone 200 mg, ibuprofen 400 mg PRN, simvastatin 40 mg, clarithromycin 500 mg b.i.d.* — the system produced five medications correctly resolved to RxNorm codes with full confidence, seven interactions across three severity tiers (five

high, one moderate, one low), and a plain-English summary at grade 10.7. The total pipeline ran in 19.3s end-to-end on the test hardware.

5.2 LLM-as-Judge Reliability

Across 17 test cases distributed over the four agents, the judge passed 14 (82% pass rate). The three failures clustered in the Coordinator suite, where the relaxation of the LLM sanity check (in favour of performance) means a small number of borderline inputs — general medical questions phrased to look like medication lists — are no longer rejected upstream. We accept this trade-off because downstream agents gracefully return empty results in those cases.

5.3 Findings

F1. Tool-grounding (RxNorm + openFDA + curated DB) is necessary and sufficient to suppress LLM hallucinations on factual claims (RxCUIs, mechanisms, severities). The judge flagged zero fabricated RxCUIs across all test cases.

F2. Iterative self-correction (Communicator’s `simplify_text` loop) is expensive at SLM cost; a tightened trigger threshold and a single-iteration cap recover most of the latency without measurably degrading output quality.

F3. Custom JSONL tracing is sufficient for production observability and enables data-driven optimisation; we identified the two largest hot-spots in under five minutes of trace analysis.

6. Conclusions and Future Work

6.1 Summary

RxSentinel demonstrates that a four-agent multi-agent pipeline — previously the preserve of frontier-model deployments — is achievable on consumer hardware in 2026 with an open-source stack. The system delivers tool-grounded, plain-English medication safety reviews in ~19s end-to-end, with full observability and explicit self-evaluation, and without any paid LLM API.

6.2 Future Work and Repository

Promising extensions include (i) multilingual patient summaries (qwen2.5 already handles Sinhala and Tamil), (ii) pill-image OCR via a local vision model, (iii) token-level streaming of the patient summary to improve perceived latency, and (iv) a signed clinician-feedback endpoint feeding a domain-specific fine-tuning dataset.

The complete implementation is publicly available at <https://github.com/ri7in/ctse-assignment-2-rxsentinel>.

References

- [1] A. Bates et al., *Adverse drug events as a cause of hospital admission*, Br. J. Clin. Pharmacol., 2020.
- [2] World Health Organization, *Medication safety in polypharmacy*, 2019.
- [3] National Library of Medicine, *RxNorm REST API documentation*, <https://rxnav.nlm.nih.gov/RxNormAPIs.html>.
- [4] U.S. Food and Drug Administration, *openFDA Drug Event Endpoint*, <https://open.fda.gov/apis/drug/event/>.
- [5] LangChain AI, *LangGraph documentation*, <https://langchain-ai.github.io/langgraph/>.
- [6] Ollama Inc., *Local model serving*, <https://ollama.com>.
- [7] J.P. Kincaid et al., *Derivation of New Readability Formulas for Navy Enlisted Personnel*, Naval Tech. Training Command, 1975.
- [8] Qwen Team, Alibaba Cloud, *Qwen2.5: A Party of Foundation Models*, 2024.